پاسخ سوال اول:

$$1 < \log n < n = \alpha n = \pi n = 1000n < n \log n < n^2 < n^3 < 2^n < n!$$

مرتبه ۱ از همه و log n کوچک تر است چون ۱ اصلا رشدی ندارد. و به ورودی که به اندازه یک مقدار n هست وابستگی ندارد. و هرچقدر ورود ی بزرگ شود. همچنان یک باقی میماند. عملیات هایی مثل چاپ کردن یا یافتن یک عنصر مشخص در آرایه از این مرتبه هستند.

Log n از n کوچک تر است چون بخاطر شیوه عمل کردن log خیلی آهسته تر از خود n رشد میکند.

مثلا اگر:

$n = 1024$

$\log_2 1024 = 10$

اینجا مقدار log برای اون اندازه از n شد ۱ درحالی که مقدار n همان ۱۰۲۴ باقی ماند.

چرا αn و πn و 1000n و n رو مساوی میگیریم باهم؟

مشخصا کاملا برابر نیستند اما از نظر مرتبه رشد که داریم بررسی میکنیم چرا برابر هستند و هر سه خطی و n حساب میشن.

ما توی تعریف Big-O بررسی میکنیم که وقتی اندازه ورودی را بزرگ کنیم. سرعت رشد به چه اندازه و شکلی بالا میرود.

مشخصا وقتی n=10 باشه:

1000n=10000

که میبینیم روی رشد اثر دارد. اما اگر ورودی رو دو برابر کنیم n=20:

1000n=20000

مثل رشد ورودی که دو برابر شده بود. دو برابر میشود.

و حتی اگر که n=100:

1000n=100000

که بازه مثل مقدار ورودی که پنج برابر شده بود. پنج برابر شد.

درواقع با تغییر اندازه n هردو دقیقا به یک نسبت رشد میکنند.

در تحلیل پیچیدگی زمانی مقدار فعلی تابع مهم نیست. چیزی که مهمه رفتار و سرعت رشدش وقتی n خیلی بزرگ میشه هست.

برای αn و πn هم همین طور. شاید شکلشون فرق کنه و گول زننده باشن ولی در نهایت یک مقدار ثابت تو دلشون دارن.

تکلیف π که مشخصه. ۳٫۱۴ و α هم درسته مقدار ثابت نداره اما مثل مثلا x عمل میکند و بهش مقدار داده میشه. محاسبه میشه و بعد به عنوان ضریب کنار n قرار میگیرد.

و در آخر همونطور که گفتیم ضریب ثابت رو نادیده میگیریم در تحلیل پیچیدگی زمانی ها به علت های که بالا توضیح داده شد و در هر سه فقط یک n باقی میماند

چرا n log n از اینا بزرگ تره؟

مشخصه که هردو n دارن ولی اگر به ضریب هاشون نگاه کنیم.

مثلا ۱۰۰۰ و log n

Log n هرچقدر هم کند باشه اما نسبت به ثابت ها چون رشدش ادام ه دار و صعودی است در بلند مدت از ثابت ها جلو میزند و تاثیر بیشتر ی دارد نسبت به ضریایب ثابت.

دلیل کوچک تر بودن n^2 از n^3 بسیار ساده و روشن هست. اگر n=20

$20^2 = 400$

$20^3 = 8000$

برای یک اندازه ورودی ثابت مثل ۲۰ رشد n^3 بیست برابر n^2 بوده!

چرا حالا 2^n از n^3 بزرگ تره؟

شاید اولش n^3 از 2^n بزرگ تر به نظر برسه اما همونطور که قبلا گفتیم ما با الگوی رشد برای ورودی ها زمانی که بزرگ بشن کار داریم نه مشاهده شهودی و تصمیم گیری صرفا بر اساس یک سری مقدار ورودی محاسبه شده.

مثلا برای n^3 اگر یک بار n=10 و n=20 و n=30 در نظر بگیریم:

$10^3 = 30$

$20^3 = 8000$

$30^3 = 27000$

اما حالا اگر همین ترتیب ورودی هارو برای 2^n اعمال کنیم:

$2^{10} = 1024$

$2^{20} = 1048576$

$2^{30} = 1073741824$

دلیلش هم اینه که n^3 سه بار مقدار n رو تو خودش ضرب میکنه

ولی 2^n در هر مرحله ۲ رو در خودش به اندازه n بار ضرب میکنه

در آخر چرا !n از همه اینها بزرگ تر هست؟

به طور خلاصه هرچه n بزرگ تر بشه تعداد زیادی عدد بزرگ در هم ضرب میشن.

مثلا اگر $n = 10$

$10! = 1 \times 2 \times 3 \times 4 \times 5 \times 6 \times 7 \times 8 \times 9 \times 10 = 3{,}628{,}800$

که به نسبت 2^n وقتی n=10 باشه. مشخصا رشد خیلی سریع تر و بدتری داره.

پاسخ سوال دوم:

الف) O(1)

فقط یک عنصر رو میخونیم و چاپ میکنیم. مهم نیست آرایه چندتا عن صر داشته باشه و چقدر بزرگ بشه. چون اصلا نیازی نیست تک تک ع ناصر رو بررسی کنیم. فقط به اون شماره index مورد نظر مراجعه می کنه و چاپش میکنه. مثل:

```
arr=[5,7,9,10,15,20]

print(arr[0])
```

خروجی میشه ۵

ب) O(n)

اگر آرایه n تا عنصر داشته باشه.الگوریتم دقیقا n برای اجرا میشه تا تک تکشون رو باهم جمع کنه. و پس تعداد عملیات ها مستقیما با n رشد میکنه یعنی اگر اندازه ورودی (n) دو برابر بشه. زمان اجرا و تعداد عملیات ها هم دو برابر میشه.مثل:

```
arr=[7,89,3,4,5,44]

sum=0

for x in arr:

    sum+=x
```

و یا اینکه معمولا هروقت یک حلقه for دیدید که n بار داره اجرا میشه میتونید بگید از مرتبه O(n) است. ولی حتما چک کنید که چطور کار میکنه. هر حلقه for الزاما O(n) نیست.

ج) O(n^2)

حالا فرض کنید n عنصر داریم و برای هر کدومشون هم باید n تا مقایسه انجام بدیم. یعنی تک تک عناصر به نوبت باهم مقایسه میشن. (عنصر اول رو با تمام عناصر مقایسه میکنیم. عنصر دوم رو با تمام عناصر مقایسه میکنیم و الی آخر...) که خب میشه $n \times n$ که میشه O(n^2) مثل:

```
arr=[2,4,6,8,8,9,1]

n=len(arr)

for i in range(n):

    for j in range (i+1 , n):

        if arr [i] != arr[j]:

            print(arr[i], arr [j])
```

میتونیم اگر دوتا حلقه for "تو در تو" هم دیدیم بگیم از مرتبه(O(n^2 هست اما حتما تاکید میشه قبلش روش کارکردش رو چک کنید.

د) O(log n)

مثلا اگر اندازه آرایه n باشد و هربار نصف بشه. یعنی اول n بعد $\frac{n}{2}$ بعد $\frac{n}{4}$ بعد $\frac{n}{8}$

تا برسه به یک. یعنی میخواهیم بدونیم n را چند بار نصف کنیم میرسیم به ۱

که در کل یعنی این فرمول: $\frac{n}{2^k} = 1$ که میتونیم بگیم: $n = 2^k$ که انگار داریم میپرسیم ۲ رو به توان چند برسونیم n میشه؟ که تعریف لگاریتم هست

مثل: $k = \log_2 n$

```
n=16
while n>1:
    n=n//2
    print(n)
```

پس میشه گفت هروقت اندازه مسله در هر مرحله با یک ضریب ثابت مثل ۲ کوچیک یا بزرگ بشه. تعداد مراحل و مرتبه الگوریتم لگاریتمی هست.

ه) O(n log n)

این گزینه یک تفاوت ظریف اما مهم با گزینه قبلی داره. درسته آرایه نصف میشه اما با هر نصف شدن یک پیمایش کامل به روی کل آرایه انجام میده. دقت کنید که نه فقط به روی بخش نصف شده. به روی کل آرایه.

مثل گزینه قبل نصف شدن آرایه رو داریم که گفتیم log n زمان میبره. اما توی این گزینه ما برای هر مرحله نصف شدن. یک پیمایش کلیه n تای هم داریم. که میشه:

$n \times \log n = O(nlogn)$

نکته: حواستون باشه که در اینجا n رو با log n جمع نمیکنیم چون پیمایش n برای هر یک از log n مرحله تکرار میشه. مثالش:

```
arr=[1,2,3,4,5,6,7,8,]
n=len(arr)
while n>1:
    for i in range (len(arr)):
        print(arr[i])
    n//=2
```

حلقه for داره هر بار کل آرایه رو پیمایش میکنه.

چون len(arr)=8 پس عملا :

for I in range(8)

حالا نکته اینه که این for داخل حلقه while قرار داره که با n//2 هر بار داره نصف میشه

و لگاریتمی میشه اما حلقه for داخلش داره هر بار که while اجرا می شه. کل آرایه اصلی رو

پیمایش میکنه. درواقع همیشه مقدار for همون ۸ باقی میمونه.

نکته: نیاز نیست حتما یک n\\=2 توی کد ببینید که متوجه بشید طول آرایه داره نصف میشه. میتونه به شکل های دیگه هم باشه. برای همین سعی کنید همیشه مراحل کار الگوریتم و روشش رو دنبال و درک کنید :)

پاسخ سوال ۳:

1: $t(n) = 2t\left(\frac{n}{2}\right) + 1$

$a = 2, \quad b = 2, \quad f(n) = 1$

$n^{\log_b a} = n^{\log_2 2} = n$

$t(n) = \theta(n)$

حالت اول است.

2: $t(n) = 9t\left(\frac{n}{3}\right) + n\sqrt{n}$

$a = 9, \quad b = 3, \quad f(n) = n\sqrt{n}$

$n\sqrt{n} = n^{\frac{3}{2}}$

$n^{\log_b a} = \log_3 9 = n^2$

$t(n) = \theta(n^2)$

حالت اول است.

3: $t(n) = 4t\left(\frac{n}{2}\right) + n^2$

$a = 4, \quad b = 2, \quad f(n) = n^2$

$n^{\log_b a} = \log_2 4 = n^2$

$t(n) = \theta(n^2 \log n)$

حالت سوم است که برابر هستن و یکیشون رو انتخاب میکنیم و بهش یک log n اضافه میکنیم.

4: $t(n) = 16t\left(\frac{n}{4}\right) + n^2 \log n$

$a = 16, \quad b = 4, \quad f(n) = n^2 \log n$

$n^{\log_b a} = \log_4 16 = n^2$

$t(n) = \theta(n^2 log^2 n)$

حالت سوم است و نکتش اینه که حتی وقتی هم که اختلاف در حد log n هست. چون ناچیز هست بازم برابر حساب میشن و اونی که در حد یک log n بزرگ تر هست رو برمیداریم و یک log n درش ضرب میکنیم.

- چرا log n و چرا ضرب؟

درخت بازگشتی در فرم مستر همیشه ارتفاعش log n هست.

ما این همه حساب کتاب میکنیم که ببینیم در آخر هزینه کل این درخت رو بخش بازگشتش تعین میکنه ($n^{\log_b a}$)؟ یا هزینه کار خارج از بازگشت f(n) ؟

اما وقتی هردو برابر باشن یا در حد فقط یک log n اختلاف داشته باشن. ارتفاع درخت (log n) درشون ضرب میشه تا هزینه کل به دست بیاد و اون log n و ضرب از اینجا میان.

نکته: در حالت های اول و دوم این ضرب با اینکه بازم درخت log n ارتفاع داره اتفاق نمیافته چون هزینه ها برابر نیستن و متفاوتن در هر بخش و خب اصلا نیازی هم نیست. مشخص میشه کدوم بخش غالبه.

5:$t(n) = 3t\left(\frac{n}{2}\right) + n^2$

$a = 3, \quad b = 2, \quad f(n) = n^2$

$n^{\log_b a} = \log_2 3 = n^{1,585}$

$t(n) = \theta(n^2)$

حالت دوم است.

6:$t(n) = 2t\left(\frac{n}{2}\right) + n^2$

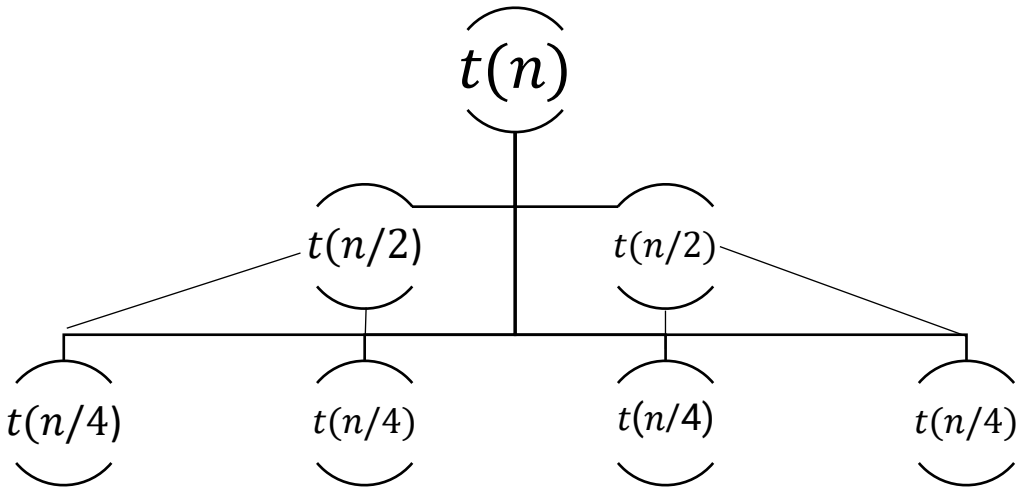
خود تابع در ابتدا:$t(n)$

توی تابع اولیه داریم: $t(n) = 2t\left(\frac{n}{2}\right) + n^2$

یعنی t(n) دوتا زیر مسله تولید میکنه:

$$t\left(\frac{n}{2}\right), \quad t(\frac{n}{2})$$

و خود این گره هم یک هزینه n^2 داره. پس:



سطح اول:

هزینه هر گره: $(\frac{n}{2})^2 = \frac{n^2}{4}$

و چون دوتا گره داریم: $\frac{n^2}{4} + \frac{n^2}{4} = \frac{2n^2}{4} = \frac{n^2}{2}$

سطح دوم:

حالا هر کدام دوباره دو شاخه شده اند: $\frac{\frac{n}{2}}{2} = \frac{n}{4}$

و چون دوباره هم هرکدام دو شاخه میشن: $4t(\frac{n}{4})$

و هزینه هر گره: $(\frac{n}{4})^2 = \frac{n^2}{16}$

که چون چهارتا تا گره داریم. پس: $4 \times \frac{n^2}{16} = \frac{n^2}{4}$

حالا سطح ام :

تعداد گره ها : 2^i

و اندازه ورودی گره: $\frac{n}{2^i}$

و همچنین هزینه هر گره: $(\frac{n}{2^i})^2 = \frac{n^2}{4^i}$

پس هزینه کل سطح : $2^i \times \frac{n^2}{4^i} = \frac{n^2}{2^i}$

بنابراین هزینه سطح ها: $n^2 + \frac{n^2}{2} + \frac{n^2}{4} + \frac{n^2}{8} + \cdots$

پس تعداد سطح ها: $k = \log_2 n$ و $n = 2^k$ و $\frac{n}{2^k} = 1$

پس جمع هزینه ها: $t(n) = n^2(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \cdots)$

و $1 + \frac{1}{2} + \frac{1}{4} + \cdots = 2$

پس: $t(n) = 2n^2$

و با ضریب ثابت کاری نداریم. پس در نهایت:

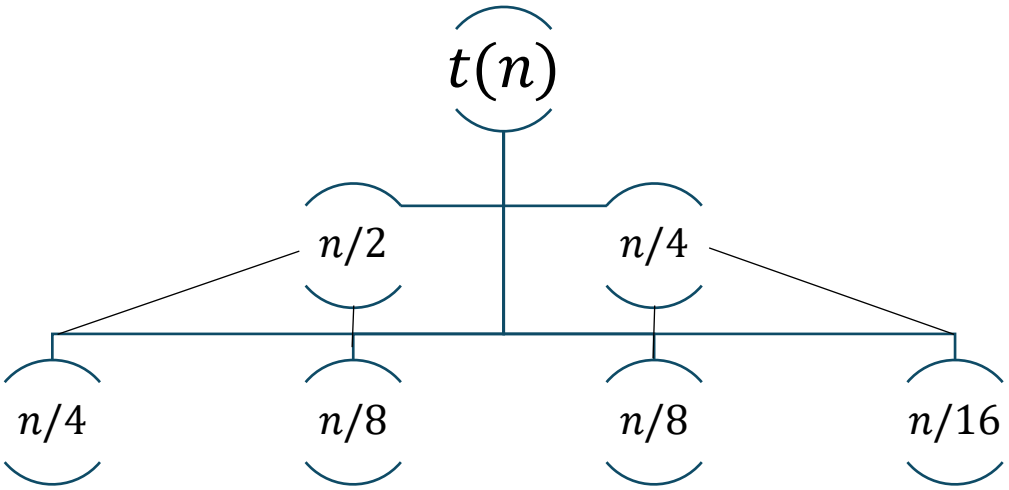
$t(n) = \theta(n^2)$

7: $t(n) = t\left(\frac{n}{2}\right) + t\left(\frac{n}{4}\right) + n$

ریشه: $t(n)$

که دوتا زیر مساله میسازه: $t\left(\frac{n}{2}\right), \quad t\left(\frac{n}{4}\right)$

هزینه n هست و درخت بازگشتیش:



- یک بار $\frac{n}{2}$ رو روی گره $\frac{n}{2}$ اجرا میکنیم که میده $\frac{n}{4}$ و یک بار $\frac{n}{4}$ رو روی $\frac{n}{4}$ اجرا میکنیم که میده $\frac{n}{8}$ و برعکس همین کار رو هم روی گره $\frac{n}{2}$ در سطح اول انجام میدیم که بهمون $\frac{n}{8}$ و $\frac{n}{16}$ میده.

هزینه سطح اول:

$$\frac{n}{2}+\frac{n}{4}=\frac{n}{2}\times\frac{2}{2}=\frac{2n}{4}+\frac{n}{4}\rightarrow\frac{2n+n}{4}=\frac{3n}{4}$$

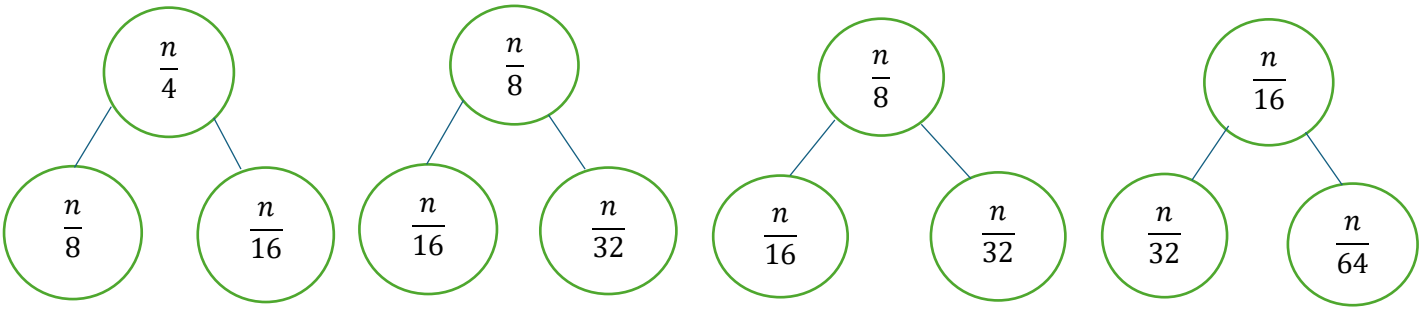
هزینه سطح دوم:

$$\frac{n}{4}+\frac{n}{8}+\frac{n}{8}+\frac{n}{16}$$

$$\frac{n}{4}=\frac{4n}{16},\quad \frac{n}{8}=\frac{2n}{16},\quad \frac{n}{8}=\frac{2n}{16},\quad \frac{n}{16}=\frac{n}{16}$$

$$\frac{4n+2n+2n+n}{16}=\frac{9n}{16}$$

سطح سوم:



هزینه سطح سوم:

$$\frac{8n}{64}+\frac{4n}{64}+\frac{4n}{64}+\frac{2n}{64}+\frac{4n}{64}+\frac{2n}{64}+\frac{2n}{64}+\frac{n}{64}\rightarrow\frac{27n}{64}$$

پس هزینه سطح ها تا الان:

$$n,\quad \frac{3n}{4},\quad \frac{9n}{16},\quad \frac{27n}{64}$$

حالا الگو مشخصه:

$$n\times\frac{3}{4}=\frac{3n}{4}$$
$$\frac{3n}{4}\times\frac{3}{4}=\frac{9n}{16}$$
$$\frac{9n}{16}\times\frac{3}{4}=\frac{27n}{64}$$

حالا هزینه سطح k میشه: $h(\frac{3}{4})^k$

داشتیم: $t(n)=n+\frac{3n}{4}+\frac{9n}{16}+\frac{27n}{64}+\cdots$

و این: $t(n)=n(1+\frac{3}{4}+(\frac{3}{4})^2+(\frac{3}{4})^3+\cdots$

که یک سری هندسی هست : $1 + r + r^2 + r^3 + \cdots + \frac{1}{1-r}$

که $r = \frac{3}{4}$ پس: $\frac{1}{1-\frac{3}{4}} = \frac{1}{\frac{1}{4}} = 4$

بنابراین: $t(n) = 4n$ که دیگه چون ۴ ثابت هست در نظر گرفته نمیشه.

در نهایت:

$t(n) = \theta(n)$

8:$t(n) = t(\sqrt{n}) + 1$

مشکل $\sqrt{n}$ هست. پس:

$n = 2^m \rightarrow \sqrt{2^m} = 2^{\frac{m}{2}}$

متغییر جدید تعریف میکنیم چون میخواهیم خود تابع روهم بر اساس m بنویسیم:

$s(m) = t(2^m)$

حالا تبدیل:

$n = 2^m$ و $\rightarrow t(n) = t(2^m)$

$\rightarrow t(2^m) = s(m)$

پس سمت چپ میشه: $s(m)$

و سمت راست میشه: $\sqrt{n} = 2^{\frac{m}{2}} \rightarrow$ و $t(\sqrt{n}) + 1$

پس: $t(\sqrt{n}) = t\left(2^{\frac{m}{2}}\right)$

و طبق تعریف s : $t\left(2^{\frac{m}{2}}\right) = s\left(\frac{3}{2}\right)$

بنابراین کل رابطه تبدیل میشه به این: $s(m) = s\left(\frac{m}{2}\right) + 1$

حالا $s\left(\frac{m}{2}\right)$ رو طبق همین رابطه باز میکنیم:

$$s\left(\frac{m}{2}\right) = s\left(\frac{m}{4}\right) + 1$$

$$s(m) = s\left(\frac{m}{4}\right) + 1 + 1$$

$$s\left(\frac{m}{4}\right) = s\left(\frac{m}{8}\right) + 1$$

$$s(m) = s\left(\frac{m}{8}\right) + 3$$

که الگوش میشه: $s(m) = s\left(\frac{m}{2^k}\right) + k$

حالا مثل هر رابطه بازگشتی دیگه. وقتی اندازه ورودی به ۱ برسه.
متوقف میشه. پس:

$$\frac{m}{2^k} = 1$$

حالا اگر دو طرف رو در یک 2^k ضرب کنیم: $m = 2^k$

حالا چون k رو میخواییم. دو طرف لگاریتم پایه ۲ میگیریم: $\log_2 m = \log_2(2^k)$

$\rightarrow m = 2^k$ و $\rightarrow \log_2(2^k) = k$ پس داریم $\rightarrow k = \log_2 m$

پس$\rightarrow s(m) = \theta(\log m)$

حالا باید برگردیم به n و حالت اولیه:

داشتیم $n = 2^m$ حالا از دو طرف لگاریتم پایه ۲ میگیریم:

$\log_2 n = \log_2(2^m)$

سمت راست: $\log_2(2^m) = m$

پس←: $\log_2 n = m$

که داشتیم : $s(m) = \theta(\log m)$ کهد اگر m رو جایگذاری کنیم:

$t(n) = \theta(\log(\log_2 n))$

9:$t(n) = t(n-1) + n$

فرض میکنیم: $t(n) = 1$

پس:

$t(2) = t(1) + 2 = 1 + 2 = 3$

$t(3) = t(2) + 3 = 3 + 3 = 6$

$t(4) = t(3) + 4 = 6 + 4 = 10$

$t(5) = t(4) + 5 = 10 + 5 = 15$

پس الگوش میشه: $1,3,6,10,15, \ldots$

حدس میزنیم:$t(n) = \frac{n(n+1)}{2}$

حالا باید با استقرای ریاضیثابنش کنم:

پایه استقرا برای n=1 سمت چپ:$t(1) = 1$

سمت راست: $\frac{1(1+1)}{2} = 1$ که درسته

حالا فرض استقرا:

فرض میکنیم برای n=k درست باشه:

$$t(k) = \frac{k(k+1)}{2}$$

ما رابطه بازگشتی رو داشتیم:

$$t(n) = t(n-1) + n$$

حالا برای اینکه بفهمیم$t(k+1)$ چیه. به جای n. K+1 میگذاریم:

$$t(k+1) = t\big((k+1)-1\big) + (k+1)$$

حالا اگر ساده کنیم:

$$t(k+1) = t(k) + (k+1)$$

حالا از فرض استقرا استفاده میکنیم:

فرض بود: $t(k) = \frac{k(k+1)}{2}$

پس در: $t(k+1) = t(k) + (k+1)$

به جای t(k) . چیزی که فرض کردیم رو میگذاریم:

$$t(k+1) = \frac{k(k+1)}{2} + (k+1)$$

$$\frac{k(k+1)}{2} + \frac{2(k+1)}{2}$$

حالا اگه یک k+1 رو ازشون فاکتور بگیریم:

$$t(k+1) = \frac{(k+1)(k+2)}{2}$$

گام استقرا:

حالا باید نشان بدیم: $t(k+1) = \frac{(k+1)(k+2)}{2}$

از رابطه بازگشتی: $t(k+1) = t(k) + k + 1$

و طبق فرض استقرا :

$$t(k+1) = \frac{k(k+1)}{2} + (k+1) = \frac{k(k+1)}{2} + \frac{2(k+1)}{2}$$

فاکتور میگیریم:

$$\frac{(k+1)(k+2)}{2}$$

که ثابت شد. پس:

$$t(n) = \frac{n(n+1)}{2}$$

و $t(n) = \frac{n^2+n}{2}$ یعنی: $t(n) = \frac{1}{2}n^2 + \frac{1}{2}n$

حالا جمله ای که سریع تر رشد میکنه رو برمیداریم. که هست: $\frac{1}{2}n^2$ که دیگه همونطور که هزار بار گفتیم ضریب ثابت رو درنظر نمیگیریم 😊

پس در نتیجه:

$$t(n) = \theta(n^2)$$

10: $t(n) = 2t(n-1) + 1$

فرض میکنیم: $t(1) = 1$

$t(2) = 2t(1) + 1 = 3$

$t(3) = 2t(2) + 1 = 7$

$t(5) = 2t(4) + 1 = 31$

پس الگوش میشه: 1,3,7,15,31

حدس میزنیم: $t(n) = 2^n - 1$

پایه استقرا : $t(1) = 1$ و $2^1 - 1 = 1$

گام استقرا ↓

رابطه بود: $t(n) = 2t(n-1) + 1$

و حدسمون بود: $t(n) = 2^n - 1$

فرض استقرا هم بود: $t(k) = 2^k - 1$

حالا تو گام استقرا باید ثابت کنیم که فرمول برای عدد بعدی. یعنی k+1 هم درست است.

$t(k+1) = 2^{k+1} - 1$

از رابطه بازگشتی استفاده میکنیم و توی رابطه اصلی به جای n میذاریم k+1

$t(k+1) = 2t\big((k+1) - 1\big) + 1$

پس: $t(k+1) = 2t(k) + 1$

حالا با استفاده از فرض استقرا:

$t(k) = 2^k - 1$

پس توی $t(k+1) = 2t(k) + 1$ به جای t(k) میذاریم:

$t(k+1) = 2(2^k - 1) + 1$

$(2 \times 2^k - 2 + 1) \rightarrow (2 \times 2^k = 2^{k+1})$

پس: $t(k+1) = 2^{k+1} - 1$ پس برای k+1 هم درست است.

بنابراین: $t(n) = 2^n - 1$

و در نتیجه : $t(n) = \theta(2^n)$

حالا اگه گفتین چرا 1- رو حساب نکردیم؟....

آفرین چون ثابته...

پاسخ سوال چهار:

الف)

- الگوریتم اول یک نسخه غیر بهینه از بابل سورت هست. دوتا حلق ه for تو در تو داره که حتی اگر آرایه مرتب هم باشه بازم کل ارایه رو پیمایش میکنه پس: $O(n^2)$

حلقه بیرونی:

for i in range(n)

هر یک دوری که این حلقه بیرونی میزنه. یک عنصر به جایگاه درستش میره.

(با عملیاتی که حلقه درونی انجام میده) که مشخصا باید جای صحیح تمام عناصر پیدا بشه پس باید برای تمام عناصر هم اجرا بشه یعنی n بار که تا اینجا هزینش میشه n .

حلقه درونی:

for j in range(0, n - i - 1)

هر بار از اول لیست شروع میکنه عناصر مجاور رو دو به دو مقایسه م یکنه و با جا به جایی آن ها بزرگ ترین عنصر باقی مونده رو به انتهای ل یست میبره که چون باید n مقایسه انجام بده پس هزینه این حلقه ه م n میشه. و چون حلقه ها هم تو در تو هستن. هزینه ها ضرب می شن: $n \times n = n^2$

- الگوریتم دوم insertion sort هست که در best case یعنی اگر آ رایه از قبل مرتب باشه. فقط شرط حلقه while اجرا میشه.
  شرطش هست:

j >= 0 and arr[j] > key

j>=0 یعنی تا زمانی که به ابتدای لیست برسی و arr[j] از key بزرگ تر با شه عملیات درون حلقه رو اجرا کن ولی در بهترین حالت که آرایه مرتب هست اصلا شرط برقرار نمیشه درسته شرط j>=0 برقرار میشه اما چون شرط arr[j]>key برقرار نیست. یعنی ارایه کناری سمت چپ key از key کوچیک تره نه بزرگ تر و برای اجرای درون حلقه باید حتما هردو شرط true باشند پس اصلا درون حلقه اجرا نمشه و فقط حلقه for اجرا

میشه که شرط while رو برای n عنصر چک میکنه. که هزینش میشه: $\Omega(n)$. اما در worst case که آرایه نا مرتب هست. چون شرط حلقه while برقرار میشه و به درون بدنه میره و حلقه "کاملا" اجرا میشه و جا به جایی صورت میگیره پس هزینه حلقه while هم در مرتبه الگوریتم لحاظ میشه که n هست و در هزینه حلقه for ضرب میشه. پس در نتیجه: $n \times n = O(n^2)$

پاسخ سوال پنجم:

الگوریتم قبل از بهینه سازی دارای مرتبه رشد$O(n^2)$ است

کارش اینه که چک کنه آیا توی آرایه جمع دو عددی وجود داره که برابر با target مورد نظر باشه یا نه؟

حلقه بیرونی میگه بیا و به اندازه طول آرایه. این حلقه درونی و عملیات هاشو انجام بده.

حالا تو دور اولش.i برابر عنصر اوله و بعد از هر دور کاملش میره عنصر بعدی

حالا حلقه درونی و عملیات هاش چیه؟

```
for j in range(i + 1, len(arr)):
    if arr[i]+arr[j]==target:
```

یعنی بیا عنصری که حلقه بیرونی (i) انتخاب کرده رو بگیر و با عنصر های بعدیش جمع کن.

اگر برابر اونtarget مورد نظرمون بود true و اگر نبودfalse بده. تا زمانی که لیست تمام بشه و بعد دوباره عدد بعدی و دوباره این مراحل حلقه for داخلی رو روش انجام بده.

حالا چرا این غیر بهینس؟ چون هر بار برای هر عدد. برمیگرده و دوباره اونو با تک تک عناصر بعدش چک میکنه. درحالی که میتونیم اطلاعاتی از عناصر قبلی نگه داریم و سریع تر بفهمیم جفت موردنظر وجود داره یا نه.

همیشه روش های مختلف و درست زیادی در دسترسمون هست که میتونیم ازش برای بهینه سازی استفاده کنیم. اما در این پاسخنامه به چند تاشون میپردازیم تا بتونید با پاسخ خودتون مقایسش کنید.

- یکی از بهترین حالت های بهینه سازی"مثل" استفاده از set :

```
def has_pair(arr, target):
    seen = set()
    for x in arr:
        if target - x in seen:
            return True
        seen.add(x)
    return False
```

چرا این روش بهینه تره؟

با استفاده از seen یک مجموعه میسازیم که اعدادی که قبلا در آرایه دیدیم رو نگه داریم.

ولی نکته مهمش که کد رو بهینه میکنه. شرط if هست که بجای اینکه دونه دونه اعداد رو جمع بزنه تا اون تارگت مشخص رو پیدا کنه. مید

خیلی هوشمندانه میاد عنصری که برداشته رو منهای تارگت میکنه.
یعنی مثلا:

target=11

x=7

target-x

که میشه:

11-7=4

یعنی هفت باید با چهار جمع بشه که بشه ۱۱ و الان میدونه دیگه باید دنبال چی بگرده. چهار.

به طور کلی یعنی میاد فقط یک بار کل لیست رو پیمایش میکنه و هر بار که یک عددی x میشه با target منها میشه و بعد دنبال جوابش توی مجموعه seen میگرده. اگر جواب بود اونجا که True و اگر نبود. False و بعد هم عددی که این عملیات رو روش انجام داده به مجموعه seen اضافه میکنه تا برای ادامه عملیات برای باقی اعداد. مجموعه کامل باشد.

که باعث میشه ما به پیچیدگی زمانیه O(n) میرسیم اما این روش به قیمت استفاده از حافظه سریع تر شده چون عناصر دارن دونه دونه توی seen ذخیره میشن. و همونطور که در صورت سوال خواسته شده بود. این روش برای حافظه ی اضافه مناسب نیست چون در بدترین

حالت ممکنه هیچ جفتی پیدا نشه و تمام عناصر در seen ذخیره بشوند.

- در حالت بعدی با استفاده از two pointers :

```python
def has_pair(arr, target):
    arr.sort()
    left = 0
    right = len(arr) - 1
    while left < right:
        total = arr[left] + arr[right]
        if total == target:
            return True
        elif total < target:
            left += 1
        else:
            right -= 1
    return False
```

این روش فقط به شرطی کار میکنه که آرایه مرتب باشه. یعنی خط arr.sort() نباید حذف بشه یا اگر شد آرایه مرتب بهش داده بشه.

توی این روش دوتا اشاره گر داریم:

left=0

right=len(arr)-1

که یکی از اول لیست و اون یکی از آخر شروع میکنه و اگر جمعشون برابر target شد Trueمیده.

شرط حلقه while:

```
while left < right
```

این عملیات رو تا وقتی که بهم برسن محدود میکنه. و دقیقا برای اینکه این شرط درست عمل کنه و الگوریتم صحیح و کامل اجرا بشه باید آرایه از قبل مرتب باشه.

و بعد ار هر مقایسه. اشاره گر چپ رو یکی به سمت راست و اشاره گر راست رو یکی به و اگر targetپیدا نشد. اول چک میکنه که آیا حاصل جمع (total) از target کوچیک تره یا نه؟ اگر بله که اشاره گر سمت چپ رو یکی میبره جلو به سمت راست چون حاصل جمع باید بزرگ تر بشه.

```
elif total < target:
        left += 1
else:
        right-=1
```

و یا اگر حاصل جمع از مقدار هدف بزرگ تر بود. اشاره گر سمت راست رو یکی میبره عقب سمت چپ تا مقدار جمع رو کوچیک تر و به target نزدیک تر کنه.

این روش مشکل حافظه اضافه روش قبل رو نداره اما بخاطر استفاده از sort که در پایتون O(n log n) زمان میبره مرتبه بالاتری رو میده

چون خود الگوریتم به تنهای O(n) زمان میبره و به "علاوه" n log n در نهایت بهمون O(n log n) میده که مشخصا از الگوریتم اصلی که $O(n^2)$ بود سریع تره.

- این روش هم مثل روش set هست اما با dictionary :

```python
def has_pair(arr,target):
    seen={}
    for x in arr:
        if target-x in seen:
            return True
        seen[x]=True
    return False
```

دقیقا همان روش را برای پیدا کردن جمع دو عددی که برابر با target میشن پیش میگیره اما بجای seen.add(x) که باهاش عنصر دیده شده رو به مجموعه seen اضافه میکرد. میگه: seen[x]=true

یعنی داخل دیکشنری. کلید x رو با مقدار True قرار بده.

مثلا اگر x=7 :

Seen={7:True}

که این روش هم همان مشکل حافظه اضافه set را دارد. حتی استفاده از set راحت تر و ساده تر هست.

و پیچیدگی این روش هم O(n) است.

- نکته: گاهی وقتا صرفا پایین بودن رشد دلیل به بهترین بهینه سازی بودن نیست. بلکه باید حواستون به دیگر وجه ها هم مثل حافظه اضافه باشه و در اینجا شاید با فدا کردن یک لوگ ان بتونیم توی عملکرد حافظه اضافه بهتر عمل کنیم و روش two pointers رو انتخاب بهتری بدونیم. 😊
- نکته: راه حل های متفاوت در صورت صحیح بودن. بدون تاثیر در خروجی. قابل قبول هستن

توجه: برای این الگوریتم به رشدی بهتر از O(n) نمیتوان دست پیدا کرد.